

TECHNISCHE UNIVERSITÄT DRESDEN

FAKULTÄT INFORMATIK  
INSTITUT FÜR SYSTEMARCHITEKTUR  
PROFESSUR FÜR DATENBANKEN  
PROF. DR.-ING. WOLFGANG LEHNER

## Diplomarbeit

zur Erlangung des akademischen Grades  
Diplom-Informatiker

# Aktive cache-bewusste Hardware-Adaption: Eine Cache-Engine für Trie-basierte Indexstrukturen

Benjamin-Elias Probst  
(Geboren am 11. April 1996 in Potsdam)

Hochschullehrer: apl. Prof. Dr.-Ing. Dirk Habich  
Betreuer: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, 1. Juni 2026



---

**Hier Aufgabenstellung einfügen!**



---

# Selbstständigkeitserklärung

Hiermit erkläre ich, Benjamin-Elias Probst (Mat.-Nr. 4510512, geboren am 11. April 1996 in Potsdam), dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

*Aktive cache-bewusste Hardware-Adaption:  
Eine Cache-Engine für Trie-basierte Indexstrukturen*

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 1. Juni 2026

Benjamin-Elias Probst  
Mat.-Nr. 4510512



---

## **Kurzfassung**

Platzhalter – deutsche Kurzfassung folgt.

## **Abstract**

Placeholder – English abstract to follow.



## **Inhaltsverzeichnis**



# 1 Einleitung

## 1.1 Motivation

Trie-basierte Indexstrukturen sind das Rückgrat moderner Hauptspeicher-Datenbanken. Seit dem Adaptive Radix Tree (ART) von Leis et al. [?] im Jahr 2013 hat sich die Forschung in mehrere Richtungen ausdifferenziert: Trie-Höhen-Optimierung (HOT [?]), hybride  $B^+$ -Strukturen (Masstree [?],  $B^2$ -Baum [?]), succinct-Repräsentationen (CoCo-Trie [?], SuRF [?]) und selbstoptimierende Ansätze (START [?]).

Parallel dazu hat sich ein eigener Forschungskorpus zu Cache-Bewusstheit, Prefetching und Allokationsstrategien entwickelt. Heuristiken wie Cache-Sensitive Search Trees, cache-oblivious Layouts und hierarchisches Prefetching adressieren jeweils eine einzelne Achse der Cache-Hierarchie — sie wurden jedoch nie systematisch in einer einzigen, permutierten Mess-Architektur kombiniert.

Diese Arbeit schließt diese Lücke mit einer dreischichtigen Cache-Engine-Architektur (CacheEngine → ExecutionEngine → SearchEngine), deren Bausteine in vierzehn orthogonale Permutations-Achsen zerlegt sind und einen Konfigurationsraum von mehr als  $10^{11}$  Permutationen aufspannen. Als Prüfling für die Stand-der-Technik-Zugehörigkeit dient PRT-ART (ein Probst-Redirect-Tree-/ART-Hybrid) mit acht eigenen Bausteine-Schichten.

## 1.2 Problemstellung

Bestehende cache-bewusste Index-Entwürfe optimieren ein oder wenige Aspekte isoliert und berichten Ergebnisse, die sich über Veröffentlichungen hinweg schwer vergleichen lassen, da jede Arbeit eigene Datenstrukturen, Workloads und Mess-Methodiken verwendet. Es fehlt ein gemeinsamer Rahmen, der (i) solche Entwürfe in austauschbare Achsen zerlegt, (ii) publizierte Algorithmen als explizite Konfigurationen rekonstruiert und (iii) sowohl einzelne Achsen als auch ganze Algorithmen auf einer einheitlichen, fairen Basis vermisst.

## 1.3 Forschungsfragen

**FF1 (Komposition)** Welche Algorithmus-Achsen eines cache-bewussten Trie-Index lassen sich kanonisch in einer Permutationsmatrix systematisieren, sodass Stand-der-Technik-Implementierungen als Profil-Konfigurationen rekonstruiert werden können?

**FF2 (Mess-Methodik)** Wie lässt sich pro Permutation ein fairer, reproduzierbarer YCSB-Workload-Lauf durchführen, der eine profil-spezifische Workload-Auswahl (z. B. ein Range-Scan-Profil → YCSB-E) erlaubt und dabei den Mess-Overhead über einen opt-in-Experiment-Modus minimiert?

**FF3 (Prüfling-Vergleich)** Wie schneidet PRT-ART als Prüfling gegen die acht Tier-1-SOTA-Entwürfe (ART, HOT, Masstree, CoCo-Trie, START,  $B^2$ -Baum, Wormhole, SuRF) und die Tier-2/3-SOTA

unter den drei Pflicht-Messreihen A (Prüfling vs. SOTA), B (SOTA-Vergleichsbasis) und C (Merge-Punkte) ab?

## 1.4 Zielsetzung und Beiträge

Die zentralen Beiträge dieser Arbeit sind:

- Eine dreischichtige Architektur (CacheEngine → ExecutionEngine → SearchEngine) mit ABI-stabilen C++23-Modul-Schnittstellen und variadisch templatierten Hybrid-API-Spezialisierungen (1 Parameter  $\Rightarrow$  `std::vector-API`; 2 Parameter  $\Rightarrow$  `std::map-API`;  $N > 2 \Rightarrow$  `std::map<K, std::tuple`
- Eine vierzehn-Achsen-Permutationsmatrix mit `std::variant`-Bausteinen und Compile-Time-Fallback zwischen Prüfling-Stack und Stand-der-Technik-Stack.
- Dreißig SOTA-Algorithmus-Profile als persistierte XML-Konfigurationen (8 Tier-1 + 22 Tier-2/3), die der `ExperimentDriver` automatisch in vorkompilierte C++23-Module überführt.
- PRT-ART als Prüfling mit acht eigenen Bausteine-Schichten (4+2-Pool-Allokator, optimistisches Lock-Coupling mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, Density-Tracker, Hypothesen-Metriken H1/H2/H3, Inline/External/ChainRef-ValueHandles, Signaling-Bit-Serialisierung).
- Ein Mess-Driver mit Defined-/Full-Modus-Unterscheidung und profil-bewusstem Workload-Routing.

## 1.5 Aufbau der Arbeit

Kapitel ?? führt in die nötigen Grundlagen ein. Kapitel ?? fasst den Stand der Technik über sechs Such-Cluster (A–F) und einen Allokator-Cluster zusammen. Kapitel ?? stellt das Achsen-Bibliotheks-Framework, die geschichtete Architektur, die ABI-Schnittstelle und die vierzehn-Achsen-Permutationsmatrix vor. Kapitel ?? dokumentiert die Implementierung über die drei Repositorien. Kapitel ?? beschreibt die Mess-Methodik mit den drei Pflicht-Messreihen und dem profil-bewussten Workload-Routing. Kapitel ?? präsentiert und diskutiert die Ergebnisse einschließlich der Hypothesen H1–H3. Kapitel ?? schließt mit Zusammenfassung und Ausblick.

## 2 Grundlagen

Dieses Kapitel schafft die für den Rest der Arbeit nötigen Grundlagen: die Cache-Hierarchie und den Begriff der Cache-Bewusstheit, trie-basierte Indexstrukturen mit dem Adaptive Radix Tree als durchgängiger Referenz, das einheitliche `std::map`-Vergleichsinterface, die YCSB-Workloads und die C++23-Metaprogrammierungs-Mittel, auf denen die Cache-Engine aufbaut.

### 2.1 Cache-Hierarchie und Cache-Bewusstheit

Moderne Prozessoren schalten mehrere Cache-Ebenen (L1, L2, L3) zwischen die Register und den Hauptspeicher. Daten werden zwischen diesen Ebenen in festen Einheiten übertragen, den *Cache-Lines*, typischerweise 64 Byte. Ein Zugriff, der sein Datum in einer Cache-Ebene findet, ist günstig; ein Fehlzugriff (Miss) wird an die nächste, langsamere Ebene weitergereicht, und ein Miss im Last-Level-Cache erreicht den Hauptspeicher zu Kosten von hunderten Zyklen. Die Adressübersetzung fügt eine zweite Hierarchie hinzu: der *Translation Lookaside Buffer* (TLB) cacht Virtuell-zu-Physisch-Abbildungen, und ein dTLB-Miss ist seinerseits teuer. Auf Mehr-Sockel-Systemen macht der nicht-uniforme Speicherzugriff (NUMA) die Latenz einer Referenz davon abhängig, welcher Knoten die Seite besitzt.

Eine Datenstruktur ist *cache-bewusst*, wenn ihr Layout auf konkrete Cache-Parameter (Line-Größe, Ebenen-Kapazitäten) abgestimmt ist, und *cache-oblivious*, wenn sie über alle Ebenen hinweg gute Lokalität erreicht, ohne sie zu kennen. Beide Philosophien tauchen im Stand der Technik (Kapitel ??) wiederholt auf und motivieren die Layout-, Prefetch- und Hardware-Achsen der vorgeschlagenen Engine.

### 2.2 Trie-basierte Indexstrukturen

Ein *Trie* (Digital-/Radix-Baum) indiziert Schlüssel anhand ihrer Byte-Folge statt durch Vergleich: jede Ebene unterscheidet anhand eines oder mehrerer Schlüssel-Bytes, sodass die Länge des Suchpfads von der Schlüssellänge abhängt, nicht von der Anzahl gespeicherter Schlüssel. Der *Verzweigungsgrad* (Fanout) und die *Knotenrepräsentation* dominieren sowohl Platzbedarf als auch Cache-Verhalten. Der Adaptive Radix Tree (ART) [?] passt die Knotenrepräsentation an den tatsächlichen Verzweigungsgrad an, nutzt dazu vier Knotentypen (Node4, Node16, Node48, Node256) und wendet Pfadkompression (Lazy Expansion, Präfix-Faltung) an, um den Baum flach zu halten. ART dient in dieser Arbeit durchgängig als Referenzentwurf.

### 2.3 Das einheitliche Vergleichsinterface (`std::map`)

Um strukturell verschiedene Suchalgorithmen fair zu vergleichen, werden sie alle auf eine einzige, gut verstandene Schnittstelle abgebildet: den C++-Vertrag der geordneten Map `std::map<Key, Value>` (`find/insert/erase/Bereichs-Iteration`). Die in Kapitel ?? eingeführten Achsen beschreiben das

*innere* Verhalten hinter dieser Schnittstelle, während die Schnittstelle selbst fest bleibt — der Vergleich zweier konformer `std::map`-Implementierungen ist somit der zentrale Mess-Akt. Eine variadische Spezialisierung hält die Schnittstelle ergonomisch: ein Template-Parameter stellt eine `std::vector`-artige API bereit, zwei Parameter die `std::map`-API und  $N > 2$  eine `std::map<K, std::tuple<...>>`.

## 2.4 YCSB-Workloads

Der Yahoo! Cloud Serving Benchmark (YCSB) definiert eine Familie von Schlüssel-Wert-Workloads, die hier als gemeinsames Lastprofil dienen: YCSB-A (50/50 Lesen/Aktualisieren, Zipf-verteilt), YCSB-B (95/5 Lesen/Aktualisieren), YCSB-C (nur Lesen), YCSB-D (jüngste Werte), YCSB-E (kurze Bereichsscans) und YCSB-F (Read-Modify-Write). Jedes Algorithmus-Profil deklariert einen erwarteten Workload (Kapitel ??), den der Mess-Driver für das profil-bewusste Routing nutzt (Kapitel ??).

## 2.5 C++23-Metaprogrammierung

Die Engine stützt sich auf Komposition zur Übersetzungszeit, um Laufzeit-Dispatch im Hot-Path zu vermeiden. *Concepts* schränken die Bausteine-Schnittstellen ein; das *Curiously Recurring Template Pattern* (CRTP) liefert statischen Polymorphismus; `if constexpr` und `std::conditional_t` wählen pro Achse zwischen einer Prüfling-Implementierung und einem Cache-Engine-Standard; und *C++23-Module* bilden eine ABI-stabile Grenze zwischen den Permutations-Binaries und dem Builder. Zusammen realisieren diese Mittel die “zero-cost abstraction”, auf der die Permutationsmatrix beruht.

## 3 Stand der Technik

Der Stand der Technik bei cache-bewussten Trie-Indexstrukturen gliedert sich in sechs thematische Such-Cluster (A–F) und einen parallelen Allokator-Korpus (A01–A23). Insgesamt wurden 33 Such-Algorithmus-Paper und 21 Allokations-Paper tiefenanalysiert; die Ergebnisse liegen als persistierte XML-Konfigurationen unter `algorithm_profiles/` in der `comdare-cache-engine` vor. Die vollständigen Bausteine- und Allokator-Matrizen finden sich in Anhang ??.

### 3.1 Trie-Familie (Cluster A)

**P01 ART** [?] führt den Adaptive Radix Tree mit adaptiven Knotengrößen Node4/16/48/256 ein. **P02 HOT** [?] optimiert die Trie-Höhe über Multi-Byte-Präfixe und Bit-Vektor-Indizes. **P03 Masstree** [?] kombiniert einen Trie mit B-Bäumen und einem Permutations-Index je interner Knoten. **P04 CoCo-Trie** [?] verwendet succinct-kodierte kompakte Seiten. **P05 START** [?] erweitert ART um Selbstoptimierung der Knotenrepräsentation. P09 LOUDS (Jacobson 1989) ist die succinct-Trie-Grundlage, und **P10 SuRF** [?] nutzt LOUDS zur Bereichsfilterung.

### 3.2 Hybrid- und B<sup>+</sup>-Familie (Cluster B)

**P06 B<sup>2</sup>-Baum** [?] setzt einen zweistufigen Verzweigungsgrad für kombinierte L1+L2-Cache-Bewusstheit ein. **P07 Wormhole** [?] kombiniert eine Hash-Tabelle mit einer linearen Präfix-Sicht. P11 CSS-Baum (Rao/Ross 1999) ist der erste Cache-Sensitive Search Tree; P12 CSB<sup>+</sup>-Baum (Rao/Ross 2000) entwickelt ihn zu einem B<sup>+</sup>-Baum weiter; P13 Hankins/Patel (2003) untersucht den Effekt der Knotengröße auf die Cache-Performance.

### 3.3 Layout-Theorie (Cluster C)

P14 Samuel/Pedersen/Bonnet (2005) macht den CSB<sup>+</sup>-Baum prozessor-bewusst. P15 Graefe/Larson (2001) untersucht B-Baum-Indizes gegenüber CPU-Caches. P16 Bender/Demaine/Farach-Colton (2002) führt cache-oblivious Baum-Layout (van Emde Boas) ein; P17 Bender et al. (2005) erweitert es zu cache-oblivious B-Bäumen. P18 Saikkonen/Soisalon-Soininen (2008) und P19 (2016) entwickeln Multi-Level- und layout-invariante B-Bäume.

### 3.4 Prefetching (Cluster D und E)

P20 “B-Trees Are Back” (Mueller/Benson/Leis 2025) demonstriert modernes B-Baum-Engineering. P21 Chen/Gibbons/Mowry (2001) führt Prefetching-B<sup>+</sup>-Bäume ein; P22 Chen et al. (2002) entwickelt fraktales

Prefetching. P23 Khan (2010) macht Cache-Prefetching dynamisch adaptiv; P24 Naderan-Tahan/Sarbazi-Azad (2016) ergänzt einen adaptiven Filter auf dem Cache-Prefetcher. P25 Mahling/Weisgut/Rabl (2025) kombiniert Bereichs-Scans mit Hot-Path-Caching. P26 Zhang (FGCS 2024) und P27 Zhang (ASPLOS 2025, hierarchisches Prefetching) sind die jüngsten Schemata, und P28 Kuehn (DaMoN 2023) motiviert datengetriebene Cache-Optimierung.

### 3.5 Synchronisation (Cluster F)

P08 “ART of Practical Synchronization” (Leis 2016) führt das optimistische Lock-Coupling (OLC) für ART ein. P29 RCU (McKenney, OLS 2001) und P30 Hazard Pointers (Michael 2004) sind die kanonischen Reklamations-Schemata. P31 Ungethüm/Habich (2017) und P32 Schmidt/Habich (2025) sind TU-Dresden-spezifische Hardware-Optimierungs-Überblicke; P33 (VAMPIR-Poster, Berthold/Habich 2023) liefert die OTF2-Profiler-Integration.

### 3.6 Allokator-Forschung (A01–A23)

Die 21 Allokations-Paper sind systematisch in der Allokator-Matrix (Anhang ??) katalogisiert. Tier-1-Repräsentanten sind Hoard (A01, ASPLOS 2000), Slab (A02, USENIX 1994), Michael Lock-Free (A03, PLDI 2004), mimalloc (A04, MSR-TR-2019-18), jemalloc (A05), TCMalloc (A06), snmalloc (A07, ISMM 2019) und NUMAAlloc (A09, ISMM 2023). Tier-2/3 ergänzen CAMA (A12, RTAS 2011), StarMalloc (A13, OOPSLA 2024), Crystalline-Reklamation (A17, PLDI 2024) und PIM-malloc (A16, arXiv 2025).

### 3.7 Profil-Schema mit dem `<expected_workload>`-Tag

Für jeden SOTA-Algorithmus und jeden Allokator hält ein Profil-XML in `cache_engine/algorithm_profiles/` Metadaten fest, die Achsen-Belegung (page, node, traversal, value\_handle, concurrency, allocator, prefetch, telemetry, isa, layout, reclamation), eine Schlüssel-Wert-Signatur und einen optionalen `<expected_workload>`-Tag, der die `ExperimentDriver`-Heuristik (Kapitel ??) überschreibt. Alle 30 SOTA-Profil- und alle 10 Allokator-Profil- sind getaggt.

Die Workload-Verteilung beträgt  $13 \times$  YCSB\_C (leselastig),  $9 \times$  YCSB\_A (Zipf-verteiltes Lesen+Aktualisieren),  $6 \times$  YCSB\_E (Bereichs-Scans) und  $2 \times$  YCSB\_B (95/5 Lesen/Aktualisieren). PRT-ART selbst ist mit YCSB\_F (Read-Modify-Write, für DensityTracker-Aktualisierungen) getaggt. Tabelle ?? listet die zehn implementierten Allokator-Profil-.

### 3.8 Hardware- und Scheduling-Strategie pro Paper

Nach der N-Phasen-Erweiterung der Bausteine-Matrix (Abschnitt ??) sind zwei zusätzliche Algorithmus-Permutations-Achsen verankert:

- **Achse 12 Hardware-Strategie:** welche Hardware-Merkmale der Algorithmus aktiv nutzt — Sub-Achsen 12.1 SIMD-Familie (AVX2/AVX-512/NEON/SVE2/scalar), 12.2 Cache-Level-Targeting

Tabelle 3.1: SOTA-Algorithmus-Profile mit Workload-Affinität

| Profil                    | Paper-Ref.               | expected_workload |
|---------------------------|--------------------------|-------------------|
| art                       | P01 (Leis 2013)          | YCSB_C            |
| hot                       | P02 (Binna 2018)         | YCSB_C            |
| masstree                  | P03 (Mao 2012)           | YCSB_A            |
| coco_trie                 | P04 (Boffa 2024)         | YCSB_C            |
| start                     | P05 (Fent 2020)          | YCSB_C            |
| b2tree                    | P06 (Schmeisser 2022)    | YCSB_E            |
| wormhole                  | P07 (Wu 2019)            | YCSB_A            |
| surf                      | P10 (Zhang 2018)         | YCSB_A            |
| css_tree                  | P11 (Rao/Ross 1999)      | YCSB_C            |
| csb_tree                  | P12 (Rao/Ross 2000)      | YCSB_E            |
| hankins                   | P13 (Hankins 2003)       | YCSB_C            |
| samuel                    | P14 (Samuel 2005)        | YCSB_C            |
| graefe                    | P15 (Graefe 2001)        | YCSB_C            |
| bender_treelayout         | P16 (Bender 2002)        | YCSB_C            |
| bender_cacheoblivious     | P17 (Bender 2005)        | YCSB_C            |
| saikkonen_multilevel      | P18 (Saikkonen 2008)     | YCSB_E            |
| saikkonen_layoutinvariant | P19 (Saikkonen 2016)     | YCSB_E            |
| btreesareback             | P20 (Mueller 2025)       | YCSB_E            |
| chen_prefetch             | P21 (Chen 2001)          | YCSB_A            |
| chen_fractal              | P22 (Chen 2002)          | YCSB_A            |
| khan_adaptive             | P23 (Khan 2010)          | YCSB_A            |
| naderan_tahan             | P24 (Naderan-Tahan 2016) | YCSB_A            |
| mahling                   | P25 (Mahling 2025)       | YCSB_E            |
| zhang_fgcs                | P26 (Zhang 2024)         | YCSB_A            |
| zhang_asplos              | P27 (Zhang 2025)         | YCSB_A            |
| kuehn                     | P28 (Kuehn 2023)         | YCSB_C            |
| rcu                       | P29 (McKenney 2001)      | YCSB_B            |
| hazard_pointers           | P30 (Michael 2004)       | YCSB_B            |
| ungethuem                 | P31 (Ungethuem 2017)     | YCSB_C            |
| to_stride                 | P32 (Schmidt 2025)       | YCSB_C            |

Tabelle 3.2: Allokator-Profile mit Lizenz und Workload-Affinität

| Profil           | Familie | Lizenz        | expected_workload |
|------------------|---------|---------------|-------------------|
| hoard            | A01     | Apache-2.0    | YCSB_A            |
| michael_lockfree | A03     | BSD-3         | YCSB_B            |
| mimalloc         | A04     | MIT           | YCSB_A            |
| jemalloc         | A05     | BSD-2         | YCSB_B            |
| tcmalloc         | A06     | BSD-3         | YCSB_C            |
| snmalloc         | A07     | MIT           | YCSB_A            |
| scalloc          | A08     | BSD-3         | YCSB_A            |
| rpmalloc         | A10     | Public Domain | YCSB_C            |
| lrmalloc         | A11     | BSD-3         | YCSB_B            |
| dlmalloc         | A20     | CC0           | YCSB_C            |

(L1/L2/L3/HBM-aware), 12.3 NUMA-Strategie, 12.4 Prefetch-Hardware (PREFETCH/PREFETCHNTA/PREFETCHW), 12.5 Atomic-Instruction-Familie (CAS/LL-SC/RMW-extended).

- **Achse 13 Scheduling-Strategie:** wie Arbeit verteilt wird — Sub-Achsen 13.1 Worker-Pool-Layout (thread-per-core / work-stealing / CPU-pinning), 13.2 SIMD-Worker-Anzahl-Limit (hardwarebedingt typisch 2 von  $N$  Kernen), 13.3 Heterogeneous-Core-Dispatch (Intel-Hybrid P-/E-Kerne), 13.4 Co-Routine-Strategie, 13.5 Batch-Granularität.

Tabelle ?? zeigt eine Auswahl dieser zwei Achsen je SOTA-Paper; der Vollkatalog steht in Anhang ??.

Tabelle 3.3: Hardware- und Scheduling-Strategie pro SOTA-Paper (Auswahl)

| Paper          | Hardware-Strategie (Achse 12)                                        | Scheduling-Strategie (Achse 13)   |
|----------------|----------------------------------------------------------------------|-----------------------------------|
| P01 ART        | AVX2, L1-aware, none, CAS                                            | thread-per-core                   |
| P02 HOT        | AVX2 (Bit-Vektor), L1-aware, CAS                                     | thread-per-core                   |
| P03 Masstree   | scalar, L1-aware, CAS                                                | thread-per-core + work-stealing   |
| P05 START      | AVX2 (Multi-Byte-Span), L1-aware, CAS                                | thread-per-core                   |
| P20 LeanStore  | AVX2, HBM-aware, NUMA-aware, CAS                                     | work-stealing + hybrid            |
| P27 hp-soft    | PREFETCH (L1/L2/L3-Bundle), all-cache, CAS                           | thread-per-core + Co-Routine      |
| P28 Kuehn      | PREFETCH (scalar Counter), L1-aware, CAS                             | thread-per-core + Sampling        |
| P31 Ungethuem  | AVX-512 (HBM), HBM-aware, NUMA-aware, CAS                            | hybrid (P-/E-Kerne)               |
| P32 Schmidt    | AVX-512 (SIMD-Stride), HBM-aware, NUMA-aware, CAS                    | hybrid                            |
| <b>PRT-ART</b> | <b>AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)</b> | <b>thread-per-core + 2-SIMD-W</b> |

### 3.9 Forschungslücke

Jede der untersuchten Arbeiten optimiert eine oder wenige Achsen der Cache-Hierarchie isoliert, und die Ergebnisse werden in nicht vergleichbaren Settings berichtet. Was fehlt — und was diese Arbeit beiträgt — ist eine systematische, orthogonale *Achsen-Zerlegung* dieser Entwürfe zusammen mit einer permutierten Mess-Architektur, die sie als explizite Konfigurationen rekonstruiert und auf einer gemeinsamen Basis vermisst.

## 4 Konzept und Architektur

Dieses Kapitel präsentiert den Kernbeitrag der Arbeit: ein Achsen-Bibliotheks-Framework, das cache-bewusste Suchalgorithmen in austauschbare, orthogonale Bausteine-Achsen zerlegt, die geschichtete Engine-Architektur, die es realisiert, die ABI-stabile Modul-Schnittstelle, die vierzehn-Achsen-Permutationsmatrix, den Prüfling PRT-ART und den Builder, der die Drei-Stufen-Prüfung orchestriert.

### 4.1 Das Achsen-Bibliotheks-Framework

Die zentrale Idee besteht darin, einen Algorithmus nicht als Monolith zu behandeln, sondern als *Komposition orthogonaler Achsen*, wobei jede Achse eine Teilentscheidung eines cache-bewussten Index repräsentiert (wie Seiten angeordnet werden, wie Knoten verzweigen, wie Speicher alloziert wird, wie Nebenläufigkeit behandelt wird usw.). Ein konkreter Algorithmus ist dann ein Punkt im kartesischen Produkt aller Achsen, und ein Stand-der-Technik-Entwurf wird als eine explizite Konfiguration rekonstruiert. Die entscheidende Eigenschaft ist die *modulare Austauschbarkeit*: Forschende können eine neue Achsen-Variante isoliert untersuchen — und ihren Effekt pro Achse und für den gesamten Algorithmus messen — ohne alle anderen neu zu implementieren. Das macht den Vergleich zweier `std::map`-Implementierungen (Abschnitt ??) zu einem kontrollierten, reproduzierbaren Experiment.

### 4.2 Vier-Subsystem-Trennung (M-Modell)

Die Diplomarbeit, der Cache-Engine-Builder, die Cache-Engine und der Prüfling sind *vier formal getrennte Subsysteme* mit unterschiedlichen Verantwortlichkeiten. Die Trennung ist orthogonal zur Drei-Repository-Aufteilung (Abschnitt ??): Builder und Engine wohnen im selben Repository, sind dort aber eine ausführbare Datei bzw. eine Bibliothek.

1. **messung\_driver** (Diplomarbeit/Code) — der Auswertungs-Orchestrator (äußere Schleife): liest die drei Messreihen-XML-Konfigurationen (A/B/C), triggert pro Reihe den Builder und sammelt die Ergebnisse für den Anhang.
2. **CacheEngineBuilder** (`cache-engine/apps`) — ein autonomes Plattform-Ausmess-System, das die Sieben-Phasen-Pipeline implementiert (discover, measure, classify, publish, bind, execute, compare) und je registrierter Engine den Permutationsraum enumeriert.
3. **CacheEngine** (`cache-engine/libs`) — die Werkzeug-Bibliothek: zwölf interne Sub-Engines  $C_1$ – $C_{12}$  (Layout, Pinning, Prefetch, Coherence, Telemetry, Allocation, Migration, Encoding, Heuristik, Topologie, Scheduler, Filter) und die Cache-Strategie-Familien, bereitgestellt als C++23-Concept-API.

4. **Prüfling (PRT-ART)** (`prt-art`) — eine bei der Cache-Engine als `IExecutingEngine` registrierte Algorithmus-Implementierung; sie implementiert die Such-Engine-Schicht für jede Achse und konsumiert Cache-Engine-Dienste während ihrer eigenen Lookup-/Insert-/Scan-Operationen.

Die definierende Eigenschaft des M-Modells ist die *bidirektionale* Cache-Engine  $\leftrightarrow$  Prüfling-Beziehung: (a) die Cache-Engine instantiiert den Prüfling je Permutations-Konfiguration (Builder-Phase 5, BIND), und (b) der Prüfling ruft zur Laufzeit Cache-Engine-Dienste (Telemetry, Prefetch, Heuristik) während seiner eigenen Lookups auf (Phase 6, EXECUTE). Diese Symmetrie ermöglicht die *Multi-Prüfling*-Fähigkeit: Messreihe A vergleicht PRT-ART gegen viele SOTA-Adapter in einer einzigen Builder-Pipeline.

### 4.3 Drei-Schichten-Hierarchie

```

CacheEngine                (Schicht 1: Stand der Technik, Werkzeug-Bibliothek)
    ^  erbt
ExecutionEngine<ProcessingStrategy>  (Schicht 2: OS-Primitive)
    ^  erbt
SearchEngine<Collection, Config>      (Schicht 3: Such-Muster)
    ^  spezialisiert
PrtArtSearchEngineAdapter              (Prüfling-Anbindung)

```

**Schicht 1 (CacheEngine)** stellt Cache-Page-Topologie, Allokator-Familien, Sub-Engines und Telemetrie-Strategien als Bibliothek bereit. **Schicht 2 (ExecutionEngine)** kapselt diese zu höheren OS-Primitiven (cache-line-aligned Allokation, NUMA-konformes Read-Pin, coherence-aware Write). **Schicht 3 (SearchEngine)** bietet such-spezifische Muster (Trie-Walk, B<sup>+</sup>-Bereichs-Scan, Hot-Path-Lookup) auf den Execution-Primitiven.

### 4.4 ABI-stabiles C++23-Modul-Interface

Das ABI folgt dem Prinzip variadischer Template-Parameter: ein Parameter  $\Rightarrow$  `std::vector-API`, zwei Parameter  $\Rightarrow$  `std::map-API`,  $N > 2 \Rightarrow$  `std::map<K, std::tuple<V1...VN>>`. Komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert. Die Modulgrenze ist binär-stabil, sodass der Builder vorkompilierte Permutations-Module laden (LoadLibrary/dlopen) und den ABI-Vertrag je Modul verifizieren kann.

### 4.5 Die Bausteine-Achsen

Nach der N-Phasen-Erweiterung umfasst die Matrix **vierzehn Hauptachsen plus etwa dreißig Sub-Achsen**: (1) Page-Type, (2) Node-Type, (3) Traversal (gesplittet in 3.A SearchAlgo, 3.B Cache-Memory, 3.M Mapping), (4) ValueHandle, (5) Memory-Layout, (6) Allocator (gesplittet in 6.1 Allocation, 6.2 Reclamation, 6.3 NUMA, 6.4 Huge-Page, 6.5 Free-List), (7) Prefetch, (8) Concurrency (8.1 Pattern, 8.2 Locking-Mode), (9) ISA, (10) Measurement, (11) Telemetry, (12) Hardware-Strategie (*neu*), (13) Scheduling-Strategie (*neu*) und (14) Identity. Je Stack-Ebene enumeriert ein `std::variant` alle Konkretisierungen; das kartesische Produkt ergibt mehr als 10<sup>11</sup> Permutationen (Abschnitt ??).

## 4.6 PRT-ART als Prüfling

Der `PrtArtSearchEngineAdapter` implementiert das `SearchEngine-ABI` durch *Komposition* statt direkter Vererbung, sodass die hybride `PrtArtSearchEngine<Ts...>-API` unberührt bleibt; das Adapter-Muster ermöglicht einen Compile-Time-Fallback auf Cache-Engine-Bausteine (`resolve_baustein.hpp`) dort, wo der Prüfling eine Achse nicht überschreibt. PRT-ART steuert eigene Implementierungen in den Achsen Page, Layout, Free-List, Prefetch, Concurrency-Pattern und Measurement bei (4+2-Pool-Allokator, Distance-Estimator-Prefetch, OLC mit reservierten Wert-Blöcken sowie die H1/H2/H3-Metriken); für die übrigen Achsen verwendet er bestehende SOTA-Bausteine über die Permutations-Konfiguration wieder.

## 4.7 CacheEngineBuilder und Drei-Stufen-Prüfung

Der Builder orchestriert eine Drei-Stufen-Prüfung: **Stufe 1** (nur Cache-Engine) nutzt die Standard-Variante je Achse; **Stufe 2** (nur Prüfling) ersetzt die überschriebenen Achsen durch die Prüfling-Varianten und behält sonst die Cache-Engine-Standards; **Stufe 3** (Full Join, nicht-redundant) kombiniert die Standard- und alle Prüfling-Varianten. Jede Stufe erzeugt tausende nicht-redundante Permutations-Binaries, die zur Laufzeit jeweils als ABI-stabiles C++23-Modul geladen werden. Die Stufen bilden direkt die drei Pflicht-Messreihen aus Kapitel ?? ab.



## 5 Implementierung

Dieses Kapitel beschreibt die Implementierung thematisch (nicht als chronologisches Entwicklungsprotokoll): die Drei-Repository-Aufteilung, die SOTA- und Allokator-Adapter, die Permutations-Codegenerierung und das Flag-System, die Concept-/CRTP-Realisierung der Achsen, Nebenläufigkeit und Speicherfreigabe, die cache-kohärenz-bewusste Telemetrie und die Mess-Pipeline.

### 5.1 Drei-Repository-Architektur

Die Implementierung verteilt sich auf drei Git-Repositoryn mit klar getrennten Verantwortlichkeiten:

- `comdare-cache-engine` — **wie** gemessen wird: die Werkzeug-Bibliothek und der Builder. Kernkomponenten: der Builder (XML-Config-Parser, `generate_module_from_profile`-Codegenerierung, Permutations-Schleife, Modul-Loader und der sieben-phasige `ExperimentDriver`); die ABI-Header (`search_engine`, `execution_engine`, `baustein_variants`, `resolve_baustein`); 30 SOTA-Profil unter `algorithm_profiles/sota`; und die Test-Suites. Der Quellbaum folgt einem Pitchfork-Layout (`apps/` für ausführbare Dateien, `libs/` für Bibliotheken, `libs/common` für querschnittlichen Code).
- `comdare-prt-art` — der **Prüfling**: die hybride `PrtArtSearchEngine` (Vector/Map/Tuple-API), ihre ABI-Adapter, ihre Bausteine (4+2-Allokator-Pools, OLC mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, DensityTracker, Hypothesen-Metriken H1/H2/H3) und die prüflingseitigen Re-Implementierungen.
- `probst-Diplomarbeit-cache-engine` — **was** getestet wird: der `messung_driver`, die Experiment-Konfigurationen, die Auswertungs-Pipeline und das Manuskript.

### 5.2 Adapter für Stand-der-Technik-Algorithmen und Allokatoren

Jeder SOTA-Entwurf und jeder Allokator wird über einen dünnen Adapter integriert, der die Original-Implementierung hinter dem Engine-ABI bereitstellt. Die Adapter folgen einem einheitlichen Muster: ein `COMDARE_HAVE_<X>`-Compile-Flag schützt einen `#if defined`-Block, der die Original-API verdrahtet, mit einem sicheren Fallback (z. B. `std::malloc` oder eine typisierte Ausnahme), wenn das Original nicht verfügbar ist. Das hält den Build selbstständig und erlaubt zugleich, die Original-Quellen je Abhängigkeit zu aktivieren. Die Integration des hierarchischen Prefetching (P27) liefert zusätzlich einen Call-Graph-Analyzer zur Übersetzungszeit (`p27_bundle_finder`, ein C++23-Port des Original-Werkzeugs `hp_soft`), der Funktionen mit Subtree-Footprint oberhalb eines Schwellwerts als Bundle-Einstiegspunkte identifiziert.

### 5.3 Permutations-Codegenerierung und Flag-System

Der Builder überführt jedes Profil in ein vorkompiliertes C++23-Modul (`generate_module_from_profile`). Jede Permutation ist durch eine bit-gepackte *Permutations-Flag*-Struktur identifiziert. Mit der N-Phasen-Verfeinerung der Matrix wächst das Flag-System von 9 Bänken (~50 Bit) auf 14 Bänke mit Sub-Bank-Bitfeldern und einem insgesamt **82-Bit-Permutations-Identifizier**, der die vierzehn Achsen und ihre Sub-Achsen widerspiegelt. Ein Constraint-Filter maskiert ungültige Achsen-Kombinationen vor jeder Modulerzeugung aus.

### 5.4 Concept-/CRTP-Realisierung der Achsen

Jede Achse ist ein Topic-Verzeichnis, das ein zweischichtiges Concept (ein breites Topic-Concept und ein enges Achsen-Concept) sowie eine durch eine `requires`-Klausel abgesicherte CRTP-Basisklasse enthält. Die Prüfling-gegen-Standard-Auswahl je Achse wird zur Übersetzungszeit über `resolve_baustein.hpp` (`std::conditional_t` über Tag-Spezialisierungen) aufgelöst, sodass der Hot-Path weder `virtual`-Aufrufe noch Laufzeit-Switches enthält — die zero-cost abstraction aus Abschnitt ??.

### 5.5 Nebenläufigkeit und Speicherfreigabe

Nebenläufigkeit ist selbst eine Achse (8.1 Pattern, 8.2 Locking-Mode), realisiert über einen Concurrency-Manager, der die Disziplinen OLC, HTM, STM, Lock-Free, RCU und Hazard Pointers koordiniert. Die Speicherfreigabe (Reclamation) ist eine Sub-Achse des Allokators (6.2): epoch-basierte, RCU-, Hazard-Pointer- und QSBR-Strategien sind je Permutation wählbar. PRT-ART steuert OLC mit reservierten Wert-Blöcken als Nebenläufigkeits-Variante bei.

### 5.6 Telemetrie gegen Cache-Kohärenz-Effekte

Per-Node-Zähler verursachen auf Mehr-Kern-Systemen Cache-Line-Ping-Pong, was genau jene Messungen verfälscht, die sie erheben. Der Kuehn-Forschungslinie (P28) folgend bietet die Telemetrie-Achse daher cache-kohärenz-bewusste Strategien: *Leaf-Only-Zähler* (die Hauptvariante), *Sampling* (jeder  $N$ -te Blattzugriff) und *Offline-Recompute* (Bottom-up-Aggregation vor dem Reordering); der klassische Per-Node-Zähler bleibt nur als gekennzeichnetes Anti-Pattern zum Vergleich erhalten.

### 5.7 Mess-Pipeline

Die Auswertungs-Werkzeugkette ist eine Folge kleiner Programme: `sample_data_generator` (deterministische Testdaten ohne reale Hardware) → `messung_driver` (steuert die Reihen) → `binary_to_csv` (deserialisiert die binären Mess-Records) → `csv_to_latex` (booktabs-Tabellen mit Algorithmus-Steckbriefen pro Profil) → `diagram_generator` (A4-bewusste TikZ-Plots) → `latex_to_pdf`. Der Mess-Pfad in der Execution-Engine wird nur unter `-DCOMDARE_EXPERIMENT_MODE=ON` mit-kompiliert, sodass der Produktiv-Pfad keinen Mess-Overhead trägt. Continuous Integration (GitLab CI plus ein gespiegelter GitHub-Workflow) läuft über alle drei Repositorien.

## 6 Evaluationsmethodik

Dieses Kapitel beschreibt, wie gemessen wird; die konkreten Ergebnisse folgen in Kapitel ??.

### 6.1 Drei Messreihen

Die Architektur sieht drei Messreihen vor, die alle vom `messung_driver` über die `ExperimentDriver`-Bibliothek ausgeführt werden, konfiguriert entweder als drei feste Reihen oder über eine externe `messreihen.xml`.

**Reihe A — PRT-ART vs. Stand der Technik.** Vergleicht den Prüfling gegen die dokumentierten SOTA-Algorithmen. Zwei Modi: *A\_defined* (schnelle Verifikation gegen die 8 Tier-1-Profilen) und *A\_full* (alle 30 SOTA-Profilen; der Standard, gemäß der Direktive, so vollständig wie möglich zu messen).

**Reihe B — Cache-Engine-Permutationen.** Die reine “Stand der Technik”-Vergleichsbasis: alle SOTA-Profilen ohne den PRT-ART-Prüfling.

**Reihe C — Merge alt/neu.** Konkrete Merge-Punkte zwischen PRT-ART-Bausteinen und Cache-Engine-Permutationen, z. B. PRT-ART OLC + `tcmalloc` + ART-Page; PRT-ART 4+2-Pool + HOT-Page; PRT-ART Path-Prefetch + B<sup>2</sup>-Baum-Page; PRT-ART MultiLevel-Layout + Wormhole-Page.

### 6.2 ExperimentDriver-Phasen

Der `ExperimentDriver` durchläuft sieben Phasen je Reihe: (1) **Enumerate** (XML-Configs parsen und die Permutationsliste aufbauen); (2) **Codegen** (ein `comdare_perm_<fp>.cpp` je Permutation, plus `generate_module_from_profile` für die SOTA-Profilen); (3) **Compile**; (4) **Load** (`LoadLibrary/dlopen` aller Module); (5) **Execute** (`create_instance` + profil-bewusstes `run_workload`); (6) **Measure** (die binären Mess-Records aggregieren); (7) **Persist** (`measurements.csv` + `.json`). Zwei opt-in-Phasen kompilieren fehlende Module hot und verifizieren den ABI-Vertrag je Modul.

### 6.3 Hypothesen

**H1 (Page-Type-Kosten)** Die mittleren Zugriffskosten je Page-Type-Variante unterscheiden sich systematisch nach Workload: dichte Page-Typen werden unter Zipf-Verteilung (YCSB-A) erwartet zu gewinnen, bereichsoptimierte Pages unter Bereichs-Scans (YCSB-E).

**H2 (Code-Qualität pro Quelle)** Der vom Original-Quell-Repository geerbte Code-Qualitäts-Score korreliert messbar mit dem erreichten Durchsatz.

**H3 (Inline- vs. External- vs. Chain-Verteilung)** Die beobachtete Verteilung der drei ValueHandle-Varianten korreliert mit der Page-Dichte: dichte Pages bevorzugen Inline, spärliche Pages External und PRT-ART-Heuristiken ChainRef.

## 6.4 Workloads und Datensätze

Phase 5 leitet den YCSB-Workload je Permutation aus dem Traversal-Tag des Profils ab (Tabelle ??); Module ohne Profil verwenden den globalen Default-Workload.

Tabelle 6.1: Profil-bewusstes Workload-Routing

| Traversal-Tag               | YCSB-Workload | Charakteristik                  |
|-----------------------------|---------------|---------------------------------|
| RANGE_SCAN / RANGE_HOT_PATH | YCSB-E        | Bereichsabfragen                |
| HOT_PATH / PRTART_HOT_PATH  | YCSB-A        | 50/50 Lesen/Aktualisieren, Zipf |
| STANDARD (Default)          | YCSB-C        | nur Lesen                       |

## 6.5 Versuchsplattformen

Die Messungen zielen auf eine lokale Workstation zur Entwicklungs-Verifikation und den TU-Dresden-ZIH-Cluster (Barnard CPU, Capella GPU) für die vollständigen Sweeps, plus die Tier-3-Plattformen (Ryzen 9 9950X3D, Intel i9 14900KS und eine HBM-Plattform). Ein `IPlatformProbe` (Phase 1) meldet je Plattform die lauffähige Teilmenge der Achsen (z. B. AVX-512 nur dort, wo verfügbar, HBM-aware nur auf HBM-Hardware).

## 6.6 Permutations-Explosion und Reduktion

Mit der N-Phasen-Erweiterung von 11 auf 14 Achsen plus  $\sim 30$  Sub-Achsen wächst der Raum von  $\sim 5,5$  Milliarden auf mehr als  $10^{11}$  Permutationen. Drei Mechanismen dämpfen die Explosion: (1) der `<expected_workload>`-Profil-Filter routet nur kompatible Workloads; (2) `MessreihenMode::Defined` führt nur die deklarierten Tupel aus ( $\sim 100$ – $1000$  je Reihe), die das Manuskript speisen; (3) `Full` (und das realistischere `Full-Sampled`, z. B. jede 1000. Permutation) läuft nur auf dem ZIH-Cluster, unter Beachtung der vom Plattform-Probe gemeldeten Achsen-Kompatibilität.

## 7 Ergebnisse und Auswertung

Dieses Kapitel präsentiert die Mess-Ergebnisse. Konkrete Diagramme und Tabellen werden aus den Mess-Läufen generiert und aus den (sprachneutralen) Verzeichnissen `tikz/` und `tabellen/` eingebunden; siehe die Platzhalter unten. Zum Zeitpunkt der Erstellung sind die Methodik und die Auswertungs-Pipeline vorhanden und auf Beispieldaten verifiziert; die Läufe auf realer Hardware stehen aus (Abschnitt ??).

### 7.1 Auswertungs-Pipeline

Pro Reihen-Lauf schreibt der `messung_driver` unter `Code/_runs/<date>/<spec_id>/` die Dateien `measurements.csv` (eine Zeile je Permutation: Cycles, L1/L2/L3-Misses, Branches, Durchsatz), `measurements.json` und optionale binäre Records. Die Auswertung nutzt `binary_to_csv` (deserialisieren und um Profil-ID, Paper-Ref, Achsen anreichern), `csv_to_latex` (booktabs-Tabellen mit Steckbriefen je Profil) und `diagram_generator` (A4-bewusste TikZ-Balkendiagramme, Scatter-Plots, Heatmaps). Das Manuskript bindet diese über `\input{tikz/<spec_id>/...}` und `\input{tabellen/<spec_id>/...}` ein.

### 7.2 Messreihe A: PRT-ART vs. Stand der Technik

Die Hypothesen H1–H3 (Abschnitt ??) werden hier ausgewertet, sobald die Mess-Daten verfügbar sind.

### 7.3 Messreihe B: Cache-Engine-Permutationen

### 7.4 Messreihe C: Full Join

### 7.5 Achsen-Sensitivitätsanalyse

Anhand der Records je Permutation quantifiziert dieser Abschnitt, welche Achse am meisten zur Performance-Varianz beiträgt — die Grundlage für die Empfehlung von Standard-Konfigurationen je Workload-Klasse.

### 7.6 Diskussion

Die Ergebnisse werden vor dem Leitmotiv der Arbeit diskutiert: dem Übergang vom heuristischen zum informierten Cache-Management, d. h. ob telemetrie-getriebene, automatisch adaptierte Konfigurationen statisch gewählte übertreffen.



## 8 Fazit und Ausblick

### 8.1 Beantwortung der Forschungsfragen

**FF1 (Komposition).** Die vierzehn Permutations-Achsen (Page, Node, Traversal, ValueHandle, Memory-Layout, Allocator, Prefetch, Concurrency, ISA, Measurement, Telemetry, Hardware-Strategie, Scheduling-Strategie, Identity) spannen einen Konfigurationsraum von mehr als  $10^{11}$  Permutationen auf. Stand-der-Technik-Entwürfe werden über Algorithmus-Profil-XMLs eindeutig rekonstruiert: 30 SOTA-Profile (8 Tier-1 + 22 Tier-2/3) zeigen, dass die Achsen-Zerlegung P01–P32 trägt. Die variadische API-Spezialisierung ( $1/2/N$  Parameter auf `std::vector/std::map/Tuple`) macht sowohl algorithmus- als auch container-orientierte Muster auf einer Engine ausdrückbar.

**FF2 (Mess-Methodik).** Die Messung ist in einen Produktiv-Modus (`COMDARE_EXPERIMENT_MODE=OFF`, kein Overhead) und einen Experiment-Modus (der ResultAggregator ist integraler ExecutionEngine-Member) geteilt. Profil-bewusstes Workload-Routing wählt den Workload je Permutation aus dem Traversal-Tag und sichert so faire Vergleiche zwischen bereichsabfrage- und punktzugriffsorientierten Entwürfen.

**FF3 (Prüfling-Vergleich).** Der konkrete Vergleich von PRT-ART gegen die 30 SOTA-Profile steht nach den ersten Mess-Läufen aus (Abschnitt ??). Die Architektur ist vollständig vorbereitet: das PRT-ART-Profil, die drei Pflicht-Messreihen-Templates, der `ExperimentDriver` mit Auto-Pickup der SOTA-Profile und der ABI-konforme Adapter mit allen `std::map`-Operationen und Notify-Hooks.

### 8.2 Limitierungen

- **Modul-Bodies sind derzeit Stubs:** die Codegenerierungs-Pipeline erzeugt ABI-konforme Skelett-Module je Profil; die eigentliche Such-Algorithmus-Implementierung je Permutation bleibt für eine Folge-Phase offen.
- **Hardware-Validierung ausstehend:** die Konfiguration ist lokal grün, aber der vollständige `ctest`-Lauf und die End-to-End-messung\_driver-Läufe über die BlockAO-Plattformen (AVX2/AVX-512, ARM NEON/SVE) stehen aus.
- **Ergebnis-Kapitel:** aktuell methodisch; konkrete Messungen folgen.

### 8.3 Ausblick

Kurzfristig: der End-to-End-Lauf der drei Pflicht-Messreihen auf realer Hardware, die Generierung der Mess-Diagramme und die Vervollständigung der Modul-Bodies je Permutation. Mittel- bis langfristig:

ein GPU-Adapter (Grace-Hopper-Cluster, ZIH), die PIM-Allokator-Integration (A16) und die formale Verifikation ausgewählter Permutations-Klassen (StarMalloc-Stil, A13). Die Drei-Repository-Architektur erlaubt es künftigen Studierenden, weitere Prüfling-Algorithmen analog zu PRT-ART einzubringen, ohne die Cache-Engine-Werkzeug-Bibliothek zu modifizieren — ein zentrales Designprinzip für die Skalierbarkeit der Forschungsinfrastruktur.

## **Abbildungsverzeichnis**



## Tabellenverzeichnis



## **A Vollständige Messreihen**



## **B Code-Struktur**



## **C Glossar**



## **D Bausteine-Matrix**



## **E Architekturentscheidungen**



## Danksagung

Placeholder acknowledgments.

